

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет имени
академика С.П. Королева»

Институт информатики и кибернетики Кафедра
технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Выполнил: Булавкина Виктория Станиславовна

Группа: 6201-120303D

Проверил: преподаватель Борисов Д.С.

Самара, 2025

Задание 2

```
1 package functions;
2
3 public class FunctionPointIndexOutOfRangeException extends IndexOutOfRangeException { 14 usages vik
4     public FunctionPointIndexOutOfRangeException() { 6 usages viktoriabulavkina7-sys
5         super();
6     }
7
8     public FunctionPointIndexOutOfRangeException(String message) { 17 usages viktoriabulavkina7-sys
9         super(message);
10    }
11 }
```

```
FunctionPoint.java  FunctionPointIndexOutOfRangeException.java  InappropriateFunctionPointException.java x
1 package functions;
2
3 public class InappropriateFunctionPointException extends Exception { 21 usages
4     public InappropriateFunctionPointException() { 8 usages
5         super();
6     }
7
8     public InappropriateFunctionPointException(String message) { 16 usages
9         super(message);
10    }
11 }
```

Созданы два специализированных класса исключений в пакете functions:

- `FunctionPointIndexOutOfRangeException` — исключение выхода за границы набора точек при обращении к ним по номеру, наследуется от класса `IndexOutOfRangeException`;
- `InappropriateFunctionPointException` — исключение, возникающее при попытке добавить или изменить точку функции ненадлежащим образом.

Задание 3

```

public ArrayTabulatedFunction(double leftX, double rightX, double[] values) { 6 usages
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница должна быть меньше правой границы: " + leftX +
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Количество баллов должно быть не менее 2: " + values.length)
    }

    this.pointsCount = values.length;
    this.points = new FunctionPoint[pointsCount + 5];
    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, values[i]);
    }
}

public ArrayTabulatedFunction(FunctionPoint[] points) { 6 usages
    if (points.length < 2) {
        throw new IllegalArgumentException("Требуется как минимум 2 точки");
    }

    this.pointsCount = points.length;
    this.points = new FunctionPoint[pointsCount + 5];
    for (int i = 0; i < pointsCount; i++) {
        this.points[i] = new FunctionPoint(points[i]);
    }
}

@Override 4 usages
public double getLeftDomainBorder() {
    return points[0].getX();
}

```

Реализация конструкторов класса ArrayTabulatedFunction с проверками корректности аргументов.

Данная проверка добавлена в начале методов getPoint, setPoint, getPointX, setPointX, getPointY, setPointY и deletePoint.

```

100 @Override 3 usages
101 public double getPointX(int index) {
102     if (index < 0 || index >= pointsCount) {
103         throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количество: " + pointsCount);
104     }
105     return points[index].getX();
106 }
107
108 @Override 1 usage
109 public void setPointX(int index, double x) throws InappropriateFunctionPointException {
110     if (index < 0 || index >= pointsCount) {
111         throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количество: " + pointsCount);
112     }
113
114     if (index > 0 && x <= points[index - 1].getX()) {
115         throw new InappropriateFunctionPointException("Точка x= " + x + " должно быть больше предыдущей точки x= " + points[index - 1].getX());
116     }
117     if (index < pointsCount - 1 && x >= points[index + 1].getX()) {
118         throw new InappropriateFunctionPointException("Точка x= " + x + " должно быть меньше следующей точки x= " + points[index + 1].getX());
119     }
120
121     points[index].setX(x);
122 }
123
124 @Override 2 usages
125 public double getPointY(int index) {
126     if (index < 0 || index >= pointsCount) {
127         throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количество: " + pointsCount);
128     }
129     return points[index].getY();
130 }
131
132 @Override 1 usage
133 public void setPointY(int index, double y) {
134     if (index < 0 || index >= pointsCount) {
135         throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количество: " + pointsCount);
136     }
137
138     points[index].setY(y);
139 }

```

```

@Override 4 usages
public void deletePoint(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количество: " + pointsCount);
    }

    if (pointsCount < 3) {
        throw new IllegalStateException("Невозможно удалить точку: требуется минимум 2 точки, текущая: " + pointsCount);
    }

    System.arraycopy(points, srcPos: index + 1, points, index, length: pointsCount - index - 1);
    pointsCount--;
}

@Override 3 usages
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    int insertIndex = 0;
    while (insertIndex < pointsCount && points[insertIndex].getX() < point.getX()) {
        insertIndex++;
    }

    if (insertIndex < pointsCount && Math.abs(points[insertIndex].getX() - point.getX()) < Double.MIN_VALUE) {
        throw new InappropriateFunctionPointException("Точка с x= " + point.getX() + " уже существует по индексу " + insertIndex);
    }

    if (pointsCount >= points.length) {
        FunctionPoint[] newPoints = new FunctionPoint[points.length + 5];
        System.arraycopy(points, srcPos: 0, newPoints, destPos: 0, pointsCount);
        points = newPoints;
    }

    System.arraycopy(points, insertIndex, points, destPos: insertIndex + 1, length: pointsCount - insertIndex);
    points[insertIndex] = new FunctionPoint(point);
    pointsCount++;
}

```

Ножницы

Снимок экрана скопирован
Автоматически сохранен
экрана.

Разметка

В методы `setPoint()` и `setPointX()` дополнительно внедрена проверка корректности значения координаты `x`, обеспечивающая её нахождение между соседними точками для сохранения монотонности области определения.

В метод `addPoint()` добавлена проверка на отсутствие точки с таким же значением `x` во избежание дублирования координат в области определения

Задание 4

```
1 package functions;
2
3 public class LinkedListTabulatedFunction implements TabulatedFunctionImp { 7 usages
4     // Внутренний класс для узлов списка
5     private static class FunctionNode { 28 usages
6         private FunctionPoint point; 4 usages
7         private FunctionNode prev; 2 usages
8         private FunctionNode next; 2 usages
9
10        public FunctionNode(FunctionPoint point) { 2 usages
11            this.point = point;
12        }
```

Основной класс `LinkedListTabulatedFunction` содержит поля для управления списком: `head` представляет фиктивный узел циклического списка, `pointsCount` хранит количество точек для избежания пересчета, `lastAccessedNode` и `lastAccessedIndex` реализуют механизм кэширования последнего доступа для оптимизации операций.

```
// Поля основного класса
private FunctionNode head; // Голова списка (не содержит данных) 17 usages
private int size; // Количество значащих элементов 21 usages
private FunctionNode lastAccessedNode; // Для оптимизации доступа 3 usages
private int lastAccessedIndex; // Индекс последнего доступного узла 7 usages
```

Конструкторы класса полностью соответствуют `ArrayTabulatedFunction` и выполняют необходимые проверки параметров, включая проверку что левая граница меньше правой и количество точек не менее двух, после чего инициализируют список через вспомогательные методы.

```

// Метод для получения узла по индексу с оптимизацией
private FunctionNode getNodeByIndex(int index) { 6 usages
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
    }
    // Оптимизация: начинаем поиск с последнего доступного узла
    FunctionNode current;
    int startIndex;
    if (lastAccessedIndex != -1 && Math.abs(index - lastAccessedIndex) < Math.min(index, size - index)) {
        // Ближе к последнему доступному узлу
        current = lastAccessedNode;
        startIndex = lastAccessedIndex;
    }
    else if (index < size / 2) {
        // Ближе к началу
        current = head.getNext();
        startIndex = 0;
    }
    else {
        // Ближе к концу
        current = head.getPrev();
        startIndex = size - 1;
    }

    // Поиск нужного узла
    if (index > startIndex) {
        for (int i = startIndex; i < index; i++) {
            current = current.getNext();
        }
    }
    else if (index < startIndex) {
        for (int i = startIndex; i > index; i--) {
            current = current.getPrev();
        }
    }
}

```

```

80         else if (index < startIndex) {
81             for (int i = startIndex; i > index; i--) {
82                 current = current.getPrev();
83             }
84         }
85         // Сохраняем для будущей оптимизации
86         lastAccessedNode = current;
87         lastAccessedIndex = index;
88         return current;
89     }

```

Метод `getNodeByIndex` реализует оптимизированный доступ к элементам списка с использованием двухуровневой оптимизации: сначала проверяется возможность доступа от последнего `accessed` узла, затем выбирается оптимальное направление обхода - с начала или с конца списка в зависимости от позиции искомого элемента.

```
// Метод для добавления узла по индексу
private FunctionNode addNodeByIndex(int index) { 2 usages
    if (index < 0 || index > size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
    }
    FunctionNode newNode = new FunctionNode(new FunctionPoint(x: 0, y: 0));
    if (size == 0) {
        // Первый элемент
        newNode.setNext(head);
        newNode.setPrev(head);
        head.setNext(newNode);
        head.setPrev(newNode);
    }
    else if (index == size) {
        // Вставка в конец
        FunctionNode last = head.getPrev();
        last.setNext(newNode);
        newNode.setPrev(last);
        newNode.setNext(head);
        head.setPrev(newNode);
    }
    else {
        // Вставка в середину
        FunctionNode current = getNodeByIndex(index);
        FunctionNode prevNode = current.getPrev();
        prevNode.setNext(newNode);
        newNode.setPrev(prevNode);
        newNode.setNext(current);
        current.setPrev(newNode);
    }
}
```

Метод `addNodeByIndex` обеспечивает вставку нового узла в указанную позицию списка с корректным обновлением связей между узлами и поддержанием актуальности кэшированной информации о последнем доступе.

```

private void deleteNodeByIndex(int index) { 1 usage & Dorofeeva Alexandra *
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsExpection("Индекс " + index + " вне диапазона точек [0, " + (pointsCount - 1) + "]");
    }
    if (pointsCount <= 2) {
        throw new IllegalStateException("Нельзя удалить точку - останется меньше двух точек");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    nodeToDelete.prev.next = nodeToDelete.next;
    nodeToDelete.next.prev = nodeToDelete.prev;
    pointsCount--;

    // Обновляем кэш
    if (lastAccessedNode == nodeToDelete) {
        lastAccessedNode = (index < pointsCount) ? nodeToDelete.next : head.next;
        lastAccessedIndex = (index < pointsCount) ? index : 0;
    } else if (lastAccessedIndex > index) {
        lastAccessedIndex--;
    }
}

```

Метод `deleteNodeByIndex` выполняет удаление узла по указанному индексу с проверкой что после удаления останется минимально необходимое количество точек, а также корректно обновляет кэш последнего доступа при необходимости.

Задание 5

```
145 // Конструкторы
146 @ public LinkedListTabulatedFunction(double[] xValues, double[] yValues) { 28 usages
147     if (xValues.length != yValues.length) {
148         throw new IllegalArgumentException("Массивы должны иметь одинаковую длину");
149     }
150     if (xValues.length < 2) {
151         throw new IllegalArgumentException("Требуется как минимум 2 точки");
152     }
153
154     initializeList();
155     for (int i = 0; i < xValues.length; i++) {
156         addNodeToTail().setPoint(new FunctionPoint(xValues[i], yValues[i]));
157     }
158 }
159 @ public LinkedListTabulatedFunction(FunctionPoint[] points) { 22 usages
160     if (points.length < 2) {
161         throw new IllegalArgumentException("Требуется как минимум 2 точки");
162     }
163
164     initializeList();
165     for (FunctionPoint point : points) {
166         addNodeToTail().setPoint(point);
167     }
168 }
169
170 // Реализация методов
171 @Override 4 usages
172 @ public double getLeftDomainBorder() {
173     if (size == 0) {
174         throw new IllegalStateException("Нет точек в функции");
175     }
176     return head.getNext().getPoint().getX();
```

Конструкторы класса `LinkedListTabulatedFunction` принимают параметры левой и правой границ области определения и количество точек либо массив значений, при этом выполняют строгую проверку входных данных на корректность - убеждаются что левая граница действительно меньше правой и что количество точек соответствует минимальному требованию не менее двух, что является обязательным условием для существования табулированной функции. В случае нарушения этих условий конструкторы выбрасывают `IllegalArgumentException` с соответствующим сообщением об ошибке, после чего инициализируют внутреннее состояние объекта через вспомогательные методы `initialize` и

initializeWithValues, которые создают начальную структуру связанного списка с равномерно распределенными точками на заданном интервале.

```
186  public double getFunctionValue(double x) {
187     if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
188         return Double.NaN;
189     }
190
191     // Поиск интервала для интерполяции
192     for (int i = 0; i < size - 1; i++) {
193         FunctionNode current = getNodeByIndex(i);
194         FunctionNode next = current.getNext();
195
196         double x1 = current.getPoint().getX();
197         double x2 = next.getPoint().getX();
198         double y1 = current.getPoint().getY();
199         double y2 = next.getPoint().getY();
200
201         if (x >= x1 && x <= x2) {
202             if (x1 == x2) {
203                 return y1; // Вертикальный отрезок
204             }
205             // Линейная интерполяция
206             return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
207         }
208     }
209
210     return Double.NaN;
211 }
```

Метод getFunctionValue демонстрирует преимущества прямой работы со структурой списка путем выполнения прямого последовательного обхода для поиска интервала содержащего x , что эффективнее использования getNodeByIndex для данной операции.

```

261  public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
262     // Находим позицию для вставки
263     int insertIndex = size;
264     for (int i = 0; i < size; i++) {
265         double currentX = getPointX(i);
266         if (currentX == point.getX()) {
267             throw new InappropriateFunctionPointException("Точка с X=" + point.getX() + " уже существует");
268         }
269         if (currentX > point.getX()) {
270             insertIndex = i;
271             break;
272         }
273     }
274     FunctionNode newNode = addNodeByIndex(insertIndex);
275     newNode.setPoint(point);
276 }
277 }

```

Метод addPoint оптимизирован за счет прямого обхода списка для проверки уникальности координаты X и поиска позиции вставки, что позволяет избежать многократных вызовов getNodeByIndex и снижает сложность операции.

```

// Инициализация пустого списка с головой
private void initializeList() { 2 usages
    head = new FunctionNode(new FunctionPoint( x: 0, y: 0));
    head.setNext(head);
    head.setPrev(head);
    size = 0;
    lastAccessedNode = head;
    lastAccessedIndex = -1;
}

```

Задание 6

```
1 package functions;
2
3 public interface TabulatedFunctionImp { 14 usages 2 implementations new *
4     // Методы доступа к области определения
5     double getLeftDomainBorder(); 4 usages 2 implementations new *
6     double getRightDomainBorder(); 4 usages 2 implementations new *
7     double getFunctionValue(double x); 3 usages 2 implementations new *
8
9     // Методы работы с точками
10    int getPointsCount(); 7 usages 2 implementations new *
11    FunctionPoint getPoint(int index); 6 usages 2 implementations new *
12    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; 2 usages
13    double getPointX(int index); 3 usages 2 implementations new *
14    void setPointX(int index, double x) throws InappropriateFunctionPointException; 1 usage 2 implementations
15    double getPointY(int index); 2 usages 2 implementations new *
16    void setPointY(int index, double y); 1 usage 2 implementations new *
17    void deletePoint(int index); 4 usages 2 implementations new *
18    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; 3 usages 2 implementations
19 }
```

Создан интерфейс TabulatedFunction, определяющий контракт для работы с табулированными функциями. Интерфейс объявляет следующие методы:

- getPoint(), setPoint() - получение и модификация точек
- getPointX(), setPointX(), getPointY(), setPointY() - работа с координатами
- addPoint(), deletePoint() - добавление и удаление точек
- getLeftDomainBorder(), getRightDomainBorder() - границы области определения
- getPointsCount() - количество точек
- getFunctionValue() - вычисление значения функции»

Задание 7

Проверяем

Main:

```

1 import functions.FunctionPoint;
2 import functions.ArrayTabulatedFunction;
3 import functions.LinkedListTabulatedFunction;
4 import functions.InappropriateFunctionPointException;
5 import functions.FunctionPointIndexOutOfBoundsException;
6 import functions.TabulatedFunctionImp;
7
8 public class Main {
9     public static void main(String[] args) {
10         System.out.println("=== ТЕСТИРОВАНИЕ ARRAY TABULATED FUNCTION ===");
11         testFunction(new ArrayTabulatedFunction( leftX: 0.1, rightX: 10.1, pointsCount: 6));
12         System.out.println("\n\n=== ТЕСТИРОВАНИЕ LINKED LIST TABULATED FUNCTION ===");
13         testFunction(new LinkedListTabulatedFunction(new double[]{0.1, 2.1, 4.1, 6.1, 8.1, 10.1}, new double[]{0, 0, 0, 0, 0, 0}));
14         System.out.println("\n\n=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===");
15         testExceptions();
16     }
17
18     private static void testFunction(TabulatedFunctionImp function) {
19         System.out.println("Функция log10(x):");
20         System.out.println("Область определения: [" + function.getLeftDomainBorder() + "; " + function.getRightDomainBorder() + "]");
21         System.out.println("Количество точек: " + function.getPointsCount());
22         System.out.println("Тип функции: " + function.getClass().getSimpleName());
23         // Устанавливаем значения y = log10(x)
24         System.out.println("\nТочки функции:");
25         for (int i = 0; i < function.getPointsCount(); i++) {
26             double x = function.getPointX(i);
27             double y = Math.log10(x);
28             function.setPointY(i, y);
29             System.out.printf("Точка %d: (%.1f; %.3f)%n", i + 1, x, y);
30         }
31         System.out.println("\nЗначения функции в различных точках:");
32         double[] testPoints = {-1, 0, 0.1, 0.3, 0.5, 1, 1.5, 2, 3, 5, 7, 10, 10.1, 11};
33         for (double x : testPoints) {
34             double y = function.getFunctionValue(x);
35             if (Double.isNaN(y)) {
36                 System.out.printf("f(%.1f) = не определено%n", x);
37             }
38         }
39     }
40 }

```

```

    }
    else {
        double exact = Math.log10(x);
        System.out.printf("f(%.1f) = %.3f (точное: %.3f)%n", x, y, exact);
    }
}

// Тестирование операций с точками
try {
    System.out.println("\nДобавляем точку (3; 0.477):");
    function.addPoint(new FunctionPoint( x: 3, Math.log10(3)));
    System.out.println("Количество точек после добавления: " + function.getPointsCount());

    System.out.println("\nДобавляем точку (7; 0.845):");
    function.addPoint(new FunctionPoint( x: 7, Math.log10(7)));
    System.out.println("Количество точек после добавления: " + function.getPointsCount());

    System.out.println("\nУдаляем точку с индексом 0:");
    function.deletePoint( index: 0);
    System.out.println("Количество точек после удаления: " + function.getPointsCount());

    System.out.println("\nИтоговая информация о точках:");
    for (int i = 0; i < function.getPointsCount(); i++) {
        double x = function.getPointX(i);
        double y = function.getPointY(i);
        System.out.printf("Точка %d: (%.1f; %.3f)%n", i + 1, x, y);
    }
}
catch (Exception e) {
    System.out.println("Ошибка при работе с точками: " + e.getMessage());
}

}

private static void testExceptions() { 1 usage  @ viktoriabulavkina7-sys *
    System.out.println("1. Тестирование недопустимых индексов:");

```

```
try {
    TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0, rightX: 10, pointsCount: 5);
    func.getPoint( index: 10); // Неверный индекс
}
catch (FunctionPointIndexOutOfBoundsException e) {
    System.out.println("    Поймано исключение: " + e.getMessage());
}

System.out.println("\n2. Тестирование нарушения порядка X:");
try {
    TabulatedFunctionImp func = new LinkedListTabulatedFunction(new double[]{0.5, 1.5, 2.5}, new double[]{1, 2, 3});
    func.setPoint( index: 1, new FunctionPoint( x: 0.1, y: 5));
}
catch (InappropriateFunctionPointException e) {
    System.out.println("    Поймано исключение: " + e.getMessage());
}

System.out.println("\n3. Тестирование дублирования точек:");
try {
    TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0, rightX: 4, pointsCount: 3);
    // Добавляем точку с X=2.0, который уже существует в равномерной сетке [0, 2, 4]
    func.addPoint(new FunctionPoint( x: 2.0, y: 5.0));
} catch (InappropriateFunctionPointException e) {
    System.out.println("    Поймано исключение: " + e.getMessage());
}

System.out.println("\n4. Тестирование удаления при минимальном количестве точек:");
try {
    TabulatedFunctionImp func = new LinkedListTabulatedFunction(
        new double[]{0, 1}, new double[]{0, 1});
    func.deletePoint( index: 0); // Попытка удалить точку когда всего 2 точки
}
catch (IllegalStateException e) {
```

```

        System.out.println("    Поймано исключение: " + e.getMessage());
    }

    System.out.println("\n5. Тестирование неверных границ области определения:");
    try {
        TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 10, rightX: 0, pointsCount: 5); // Левая граница больше правой
    }
    catch (IllegalArgumentException e) {
        System.out.println("    Поймано исключение: " + e.getMessage());
    }

    System.out.println("\n6. Тестирование недостаточного количества точек:");
    try {
        TabulatedFunctionImp func = new LinkedListTabulatedFunction(new double[]{0}, new double[]{0}); // Всего 1 точка
    }
    catch (IllegalArgumentException e) {
        System.out.println("    Поймано исключение: " + e.getMessage());
    }

    System.out.println("\n7. Тестирование установки недопустимого X:");
    try {
        TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0.5, rightX: 2.5, pointsCount: 3);
        func.setPointX( index: 1, x: 0.1); // X=0.1 меньше предыдущего X=0.5 - ДОЛЖНО ВЫБРОСИТЬ ИСКЛЮЧЕНИЕ
    }
    catch (InappropriateFunctionPointException e) {
        System.out.println("    Поймано исключение: " + e.getMessage());
    }

    System.out.println("\n8. Тестирование получения точки с неверным индексом:");
    try {
        TabulatedFunctionImp func = new LinkedListTabulatedFunction(
            new double[]{0, 1, 2}, new double[]{0, 1, 2});
        func.getPointY( index: -1); // Отрицательный индекс
    }

```

```

    }
    catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("    Поймано исключение: " + e.getMessage());
    }

    System.out.println("\n9. Тестирование особых случаев вычисления функции:");
    TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 1, rightX: 10, pointsCount: 5);
    System.out.println("    f(-5) = " + func.getFunctionValue( x: -5)); // Вне области определения
    System.out.println("    f(15) = " + func.getFunctionValue( x: 15)); // Вне области определения

    System.out.println("\n10. Тестирование работы с пустой функцией:");
    try {
        // Создаем временную функцию и удаляем все точки кроме двух
        TabulatedFunctionImp tempFunc = new LinkedListTabulatedFunction(new double[]{0, 1, 2, 3}, new double[]{0, 1, 2, 3});
        tempFunc.deletePoint( index: 2);
        tempFunc.deletePoint( index: 1);
        System.out.println("    Оставшееся количество точек: " + tempFunc.getPointsCount());
        System.out.println("    Область определения: [" + tempFunc.getLeftDomainBorder() + "; " + tempFunc.getRightDomainBorder() + "]");
    }
    catch (Exception e) {
        System.out.println("    Ошибка: " + e.getMessage());
    }
}

```


C:\Users\Виктория\.jdk\openjdk-25.0.1\bin

== ТЕСТИРОВАНИЕ ARRAY TABULATED FUNCTION

Функция $\log_{10}(x)$:

Область определения: $[0.1; 10.1]$

Количество точек: 6

Тип функции: ArrayTabulatedFunction

Точки функции:

Точка 1: (0,1; -1,000)

Точка 2: (2,1; 0,322)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (8,1; 0,908)

Точка 6: (10,1; 1,004)

Значения функции в различных точках:

$f(-1,0)$ = не определено

$f(0,0)$ = не определено

$f(0,1)$ = -1,000 (точное: -1,000)

$f(0,3)$ = -0,868 (точное: -0,523)

$f(0,5)$ = -0,736 (точное: -0,301)

$f(1,0)$ = -0,405 (точное: 0,000)

$f(1,5)$ = -0,074 (точное: 0,176)

$f(2,0)$ = 0,256 (точное: 0,301)

$f(3,0)$ = 0,453 (точное: 0,477)

$f(5,0)$ = 0,690 (точное: 0,699)

$f(7,0)$ = 0,841 (точное: 0,845)

$f(10,0)$ = 1,000 (точное: 1,000)

$f(10,1)$ = 1,004 (точное: 1,004)

$f(11,0)$ = не определено

Добавляем точку (3; 0.477):

Количество точек после добавления: 7

Добавляем точку (7; 0.845):

Количество точек после добавления: 8

Удаляем точку с индексом 0:

Количество точек после удаления: 7

Итоговая информация о точках:

Точка 1: (2,1; 0,322)

Точка 2: (3,0; 0,477)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (7,0; 0,845)

Точка 6: (8,1; 0,908)

Точка 7: (10,1; 1,004)

=== ТЕСТИРОВАНИЕ LINKED LIST TABULATED FUNCTION ===

Функция $\log_{10}(x)$:

Область определения: [0.1; 10.1]

Количество точек: 6

Тип функции: LinkedListTabulatedFunction

Точки функции:

Точка 1: (0,1; -1,000)

Точка 2: (2,1; 0,322)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (8,1; 0,908)

Точка 6: (10,1; 1,004)

Значения функции в различных точках:

$f(-1,0)$ = не определено

$f(0,0)$ = не определено

$f(0,1)$ = -1,000 (точное: -1,000)

$f(0,3)$ = -0,868 (точное: -0,523)

$f(0,5)$ = -0,736 (точное: -0,301)

$f(1,0)$ = -0,405 (точное: 0,000)

$f(1,5)$ = -0,074 (точное: 0,176)

$f(2,0)$ = 0,256 (точное: 0,301)

$f(3,0)$ = 0,453 (точное: 0,477)

$f(5,0)$ = 0,690 (точное: 0,699)

$f(7,0)$ = 0,841 (точное: 0,845)

$f(10,0)$ = 1,000 (точное: 1,000)

$f(10,1)$ = 1,004 (точное: 1,004)

$f(11,0)$ = не определено

Добавляем точку (3; 0.477):

Количество точек после добавления: 7

Добавляем точку (7; 0.845):

Количество точек после добавления: 8

Удаляем точку с индексом 0:

Количество точек после удаления: 7

Итоговая информация о точках:

Точка 1: (2,1; 0,322)

Точка 2: (3,0; 0,477)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (7,0; 0,845)

Точка 6: (8,1; 0,908)

Точка 7: (10,1; 1,004)

=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===

1. Тестирование недопустимых индексов:

Поймано исключение: Индекс: 10, Количество: 5

2. Тестирование нарушения порядка X:

Поймано исключение: X должен строго возрастать

3. Тестирование дублирования точек:

Поймано исключение: Точка с $x = 2.0$ уже существует по индексу 1

4. Тестирование удаления при минимальном количестве точек:

Поймано исключение: Невозможно удалить точку – требуется минимум 2 точки

5. Тестирование неверных границ области определения:

Поймано исключение: Левая граница должна быть меньше правой границы: $10.0 \geq 0.0$

6. Тестирование недостаточного количества точек:

Поймано исключение: Требуется как минимум 2 точки

7. Тестирование установки недопустимого X:

Поймано исключение: Точка $x = 0.1$ должно быть больше предыдущей точки $x = 0.5$

8. Тестирование получения точки с неверным индексом:

Поймано исключение: Индекс выходит за пределы: -1

9. Тестирование особых случаев вычисления функции:

$f(-5) = \text{NaN}$

$f(15) = \text{NaN}$

10. Тестирование работы с пустой функцией:

Оставшееся количество точек: 2

Область определения: $[0.0; 3.0]$

Process finished with exit code 0